1 Food or Not-Food Filter

1.1 Why do we need this filter?

During our scraping of images from various restaurant websites, we discovered that many of the images are **not food-related**. This is problematic because:

1. They are **irrelevant** for our actual classification task (e.g. predicting cuisine type).
2. They **waste storage space** (e.g. restaurants with no food images can be ignored entirely).

By filtering out non-food images early in the pipeline, we improve both **accuracy** and **efficiency**.

1.2 Which data do we use?

To train our filter, we need a dataset with labeled images of **food** and **not-food**.

Luckily, we can use the Food or Not dataset, which contains a total of **50,000 images**:

- 20,000 training images labeled as *food*
- 20,000 training images labeled as *not food*
- 5,000 test images labeled as *food*
- 5,000 test images labeled as *not food*

This dataset provides a solid and balanced base for training and evaluating our model.

1.3 Which model do we use?

Our task is **binary image classification**: determine whether an image shows food or not.

The most suitable model type for image classification is a **Convolutional Neural Network (CNN)**. However, training a CNN from scratch requires large amounts of data and computational resources. To address this, we use **transfer learning**: we start with a model pre-trained on a large dataset (ImageNet) and adapt it to our specific task.

We use **ResNet18**, an 18-layer deep **Residual Network**, which is a well-known and efficient CNN architecture. It is a good trade-off between model size, inference speed, and performance, and works well even on smaller datasets.

1.4 What needs to be fine-tuned or defined on our side?

Although we start with a pre-trained model (`resnet18(weights=ResNet18_Weights.DEFAULT)`), some adjustments are necessary to tailor it to our binary classification task:

1.4.1 Required Customizations

1. **Final Layer Replacement** The original ResNet18 is trained for 1,000 ImageNet classes. We replace the final fully connected layer with a new one that outputs exactly **2 classes**: *food* and *not food*.

   ```
   model.fc = nn.Linear(model.fc.in_features, 2)
   ```

2. **Loss Function** Since this is a classification task, we use `CrossEntropyLoss`, which is standard for multi-class problems (including binary classification with logits).

3. **Optimizer** We use the **Adam optimizer** with a learning rate of 0.001 to efficiently update weights during training.

4. **Input Transformations** The input images need to match the expected format for the pre-trained model. We use the transform pipeline provided by the weights:

   ```
   transform = weights.transforms()
   ```

5. **Early Stopping** To prevent overfitting, we implement a simple early stopping mechanism. If the validation loss does not improve for 3 consecutive epochs, training stops.

6. **Data Loaders** We use `torchvision.datasets.ImageFolder` and `DataLoader` to handle the directory structure of our training and test data efficiently.

1.4.2 Optional / Configurable Parameters in CLI or Code

- `--train_dir` and `--test_dir`: Paths to the dataset folders.
- `--max_epochs`: Number of training epochs (default: 30).
- `batch_size`, `num_workers`, etc. can be tuned for performance on specific hardware.

1.5 Summary

By fine-tuning a pre-trained ResNet18 on our labeled food/not-food dataset, we can build a highly effective and computationally efficient filter to clean up our restaurants dataset before running more complex classification tasks (e.g. cuisine detection). This improves both **model accuracy downstream** and **data management overall**.